

Natural Language Processing

AI/ML Internship Report

Submitted by

Sneha Pal

Internship Organization

IIIT Kalyani

Domain

Artificial Intelligence and Machine Learning

1 Introduction to Natural Language Processing (NLP)

Natural Language Processing (NLP) is a specialized area of Artificial Intelligence (AI) concerned with enabling computational systems to process, interpret, analyze, and generate human language. Human language is inherently complex because it contains grammar, meaning, context, ambiguity, emotion, and domain-specific usage. NLP provides computational methods for converting unstructured language data into structured representations that can be analyzed by algorithms.

The importance of NLP arises from the large volume of textual and spoken information generated in digital environments. Documents, emails, messages, academic articles, reviews, and transcripts contain valuable knowledge, but this information is usually not stored in a fixed tabular format. NLP allows machines to extract patterns from such data, support decision-making, and automate language-based tasks. In academic and industrial settings, NLP forms the basis for intelligent text processing, information retrieval, language understanding, and automated analysis.

The major goals of NLP include language understanding, language generation, information extraction, semantic representation, and text classification. A well-designed NLP system should identify important units of language, preserve meaning, and produce useful outputs for downstream machine learning tasks. These goals require both linguistic knowledge and mathematical modeling.

NLP is closely related to AI and Machine Learning (ML). AI provides the broader objective of building intelligent systems, while ML provides algorithms that learn patterns from data. NLP applies these methods to language. Traditional NLP systems often used hand-written linguistic rules, whereas modern systems rely heavily on statistical learning, neural networks, and representation learning. Thus, NLP can be viewed as a practical intersection of linguistics, AI, and ML.

2 Basic Terminologies in NLP

Several fundamental terms are used throughout NLP. A **corpus** is a collection of text data used for analysis or training. For example, a set of news articles or product reviews may form a corpus. A **document** is a single unit within a corpus, such as one article, report, or review.

A **vocabulary** is the set of unique tokens found in a corpus. A **token** is a smaller unit of text, such as a word, punctuation symbol, subword, or sentence depending on the processing method. A **feature** is a numerical or categorical representation extracted from text and used by a model. Examples of features include word counts, TF-IDF scores, or embedding vectors.

Text preprocessing refers to the set of operations applied before modeling. It may include lowercasing, tokenization, punctuation handling, stopword removal, stemming, lemmatization, and vectorization. Preprocessing is important because raw text often contains noise, inconsistent formatting, and irrelevant elements that can reduce model performance.

3 Tokenization

Tokenization is the process of dividing text into smaller meaningful units called tokens. It is generally one of the first steps in an NLP pipeline because models cannot directly process a paragraph as an unstructured block of characters.

3.1 Types of Tokenization

Word tokenization splits text into individual words. For example, the sentence “NLP is useful” may be tokenized as “NLP,” “is,” and “useful.” **Sentence tokenization** divides a paragraph into sentences, which is useful in summarization and document analysis. **Subword tokenization** divides rare or complex words into smaller pieces. This is widely used in modern neural models because it helps handle unknown words and morphologically rich languages.

Tokenization appears simple, but it can be challenging. Punctuation, contractions, abbreviations, and special symbols affect token boundaries. For example, “don’t” may be treated as one token or split into “do” and “n’t” depending

on the tokenizer design.

4 Stopword Removal

Stopwords are common words that occur frequently but may contribute limited meaning in certain tasks. Examples include “the,” “is,” “and,” “of,” “to,” and “in.” Stopword removal is the process of removing such words from text before analysis.

Stopword removal is important in tasks such as keyword extraction, document classification, and information retrieval because it reduces noise and decreases feature space size. For example, in the sentence “The student is reading a book,” the words “student,” “reading,” and “book” are usually more informative than “the” and “is.”

However, stopwords removal must be used carefully. Some words that appear common may be essential for meaning. For instance, removing the word “not” from “not good” changes the sentiment completely. Therefore, the decision to remove stopwords depends on the task and dataset.

5 Stemming

Stemming is the process of reducing inflected or derived words to a root-like form. For example, “playing,” “played,” and “plays” may be reduced to “play.” The purpose is to group related word forms so that a model treats them as similar.

Stemming generally works by applying rule-based suffix removal. Popular stemming algorithms include the Porter Stemmer, Snowball Stemmer, and Lancaster Stemmer. The Porter Stemmer is widely used because it is simple and effective for English text. Snowball is an improved and multilingual version, while Lancaster is more aggressive and may produce shorter stems.

A limitation of stemming is that the resulting root may not be a valid dictionary word. For example, “studies” may become “studi.” Despite this limitation, stemming is useful in search systems and document retrieval where exact linguistic correctness is not always required.

6 Lemmatization

Lemmatization reduces a word to its correct dictionary form, called a lemma. Unlike stemming, lemmatization considers vocabulary and grammar. For example, “running” becomes “run,” “children” becomes “child,” and “better” becomes “good.”

The key difference between stemming and lemmatization is accuracy. Stemming is faster and rule-based, but it may produce non-dictionary forms. Lemmatization is slower because it often requires part-of-speech information, but it produces linguistically meaningful base forms. For example, the word “saw” can be a noun or a verb; a lemmatizer uses context to identify the correct lemma.

Lemmatization is preferred in tasks where grammatical correctness and semantic interpretation are important, such as question answering, document understanding, and semantic search.

7 Bag of Words (BoW)

Bag of Words (BoW) is a simple text representation technique in which a document is represented by the frequency of words in a vocabulary. It ignores grammar and word order, treating a document as a “bag” of terms.

7.1 Feature Vector Representation and Worked Example

Consider two short documents: D_1 : “NLP is useful” and D_2 : “NLP is powerful.” The vocabulary is {NLP, is, useful, powerful}. The BoW vector for D_1 is [1, 1, 1, 0], and the vector for D_2 is [1, 1, 0, 1]. Each position corresponds to the

count of a vocabulary word.

BoW is easy to implement and works well for basic classification tasks. However, it has limitations. It ignores word order and context, so sentences with different meanings can have similar vectors. It also produces sparse, high-dimensional vectors when the vocabulary is large.

8 TF-IDF (Term Frequency–Inverse Document Frequency)

TF-IDF is a statistical weighting method that measures the importance of a term in a document relative to a corpus. It improves upon simple word counts by reducing the weight of common words and increasing the weight of terms that are distinctive.

The TF-IDF score for a term t in document d is commonly written as:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log \left(\frac{N}{\text{DF}(t)} \right),$$

where N is the total number of documents and $\text{DF}(t)$ is the number of documents containing term t .

For example, if the word “transformer” appears frequently in one NLP document but rarely in other documents, it receives a high TF-IDF score. This makes TF-IDF useful for document ranking, keyword extraction, and text classification. Its limitation is that it still does not capture deep semantic meaning or word order.

9 Word Embeddings: Word2Vec

Word embeddings represent words as dense numerical vectors in a continuous vector space. Unlike BoW and TF-IDF, embeddings capture semantic similarity by learning from context. Words used in similar contexts tend to have similar vector representations.

Word2Vec is a popular embedding method. It has two main architectures: Continuous Bag of Words (CBOW) and Skip-Gram. In **CBOW**, the model predicts a target word using surrounding context words. For example, given nearby words such as “cat,” “on,” and “mat,” the model may predict “sits.” In **Skip-Gram**, the model works in the opposite direction: it predicts surrounding context words from a target word.

Word2Vec has important advantages over traditional representations. It produces compact dense vectors, captures semantic relationships, and supports similarity-based reasoning. For example, related words such as “king” and “queen” can have vectors that are close in the embedding space.

10 Named Entity Recognition (NER)

Named Entity Recognition (NER) is the task of identifying and classifying named entities in text. Common entity types include Person, Organization, Location, Date, Time, Money, Percentage, Product, and Event. For example, in the sentence “Sneha visited IIT Kalyani in May,” NER can identify “Sneha” as a Person, “IIT Kalyani” as an Organization, and “May” as a Date.

The working process of NER usually involves tokenization, feature extraction or contextual embedding generation, sequence labeling, and entity classification. Earlier NER systems used rule-based patterns and gazetteers. Modern systems use machine learning and deep learning models that consider context, allowing them to distinguish entity meanings more accurately.

Example use cases of NER include extracting names from academic documents, identifying organizations in reports, detecting locations in news articles, and extracting dates or monetary values from financial text. In an AI/ML internship context, NER is significant because it transforms unstructured text into structured information suitable for analysis.

The accuracy of NER depends on the quality of training data, the clarity of entity boundaries, and the ability of the model to understand context. For instance, the word “Apple” may refer to a fruit in one sentence and a company

in another. Contextual models are useful because they examine surrounding words before assigning an entity label. This makes NER an important example of how NLP combines linguistic structure with statistical learning.

In a complete NLP workflow, the techniques discussed in this report are often connected. Tokenization prepares the input, stopword removal and normalization reduce noise, stemming or lemmatization standardizes word forms, and vectorization methods such as BoW, TF-IDF, or Word2Vec convert text into numerical features. These features can then be used by machine learning models for classification, retrieval, clustering, or entity recognition. Thus, each concept contributes to the broader objective of making natural language computationally usable.